

Ex No:1 Working with Microcontrollers

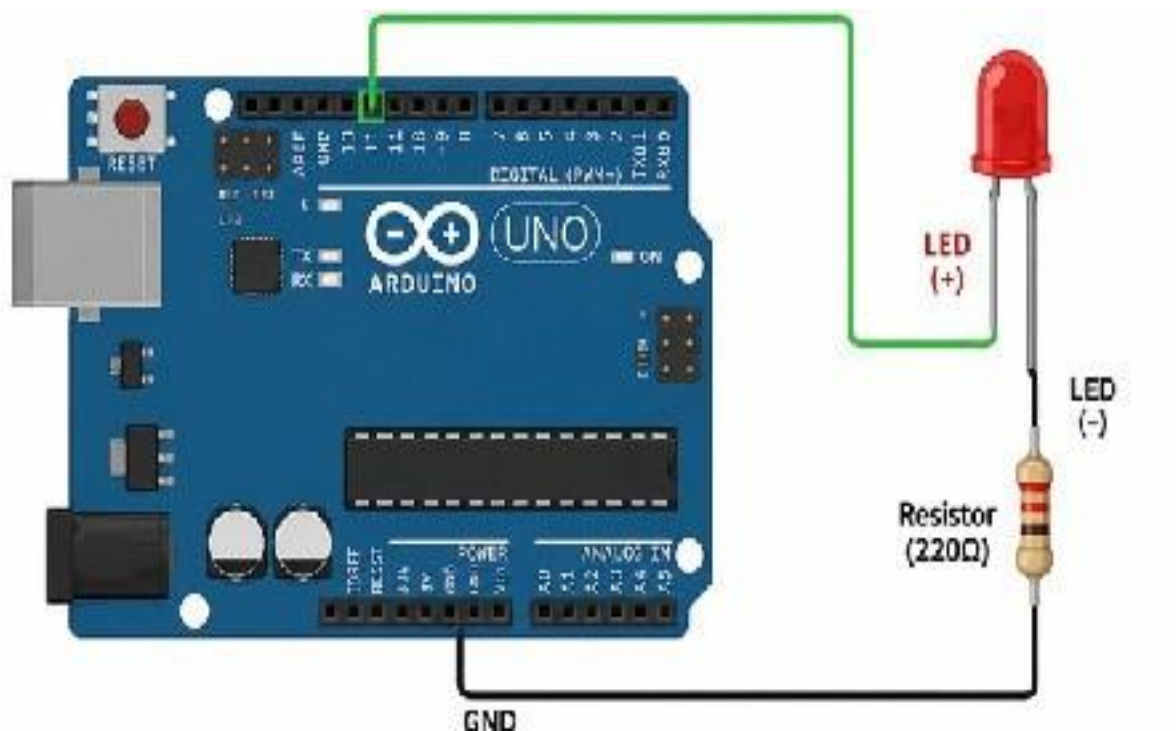
Aim

To study the working of a microcontroller and perform basic LED blinking using Arduino Uno.

Apparatus Required

- Arduino Uno board
- USB cable
- LED
- 220Ω resistor
- Breadboard
- Jumper wires
- Computer with Arduino IDE

Circuit Diagram



Procedure

1. Connect Arduino Uno to the computer.
2. Open Arduino IDE.
3. Write the program for LED blinking.

4. Select the correct COM port and board.
5. Upload the program to Arduino.
6. Observe the LED behavior.

Program

```
void setup() {  
  pinMode(13,  
    OUTPUT);  
}  
  
void loop() {  
  digitalWrite(13,  
    HIGH);  
  delay(1000);  
  digitalWrite(13,  
    LOW);  
  delay(1000); }
```

Output:

- The LED connected to Pin 13 of the Arduino Uno blinks continuously.
- The LED turns ON for 1 second and OFF for 1 second repeatedly.

Result

The working of the microcontroller and GPIO output operation was successfully verified.

EX NO 2: Collection and Data Transfer using Microcontrollers

Aim

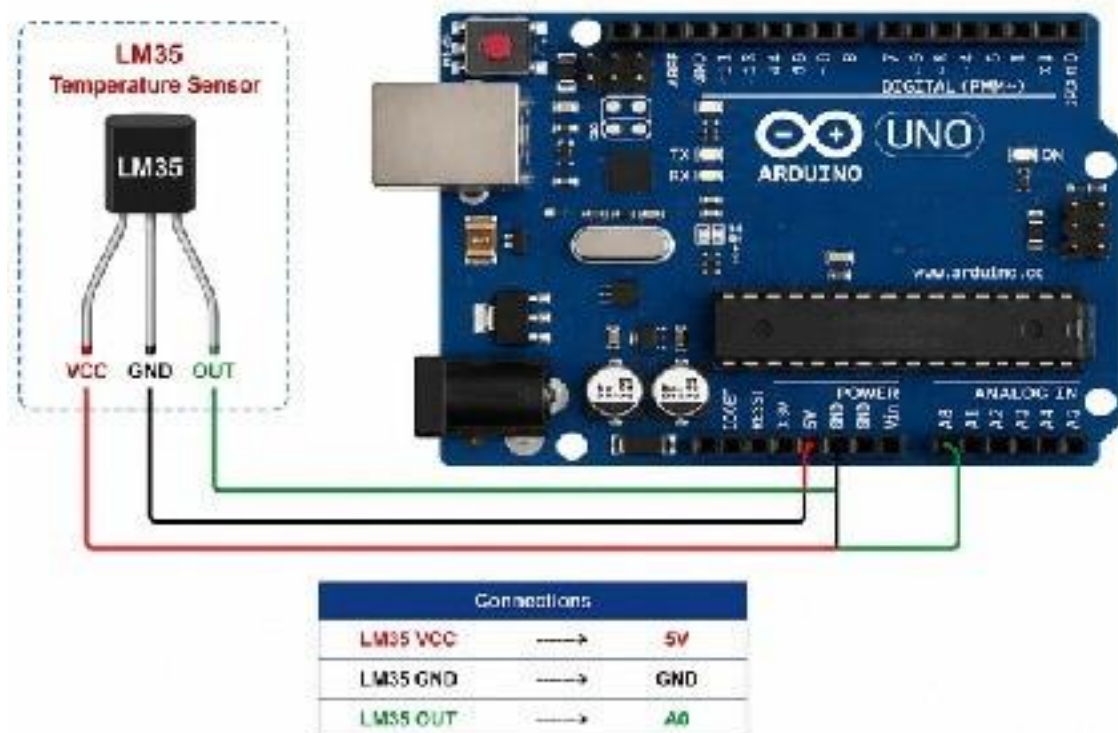
To collect sensor data and transfer it through serial communication.

Apparatus Required

- Arduino Uno
- LM35 temperature sensor
- Breadboard
- Jumper wires
- USB cable

Circuit Diagram

Arduino UNO with LM35 Temperature Sensor



Procedure

1. Connect LM35 sensor to Arduino.
2. Open Arduino IDE.
3. Upload the program.
4. Open Serial Monitor.
5. Observe transmitted temperature values.

Program

```
int sensorPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int value =
  analogRead(sensorPin); float
  voltage = value * (5.0 /
  1023.0); float temp = voltage
  * 100; Serial.println(temp);
  delay(1000);
}
```

Output:

```
Temperature: 28.90 °C
Temperature: 29.15 °C
Temperature: 29.32 °C
Temperature: 29.48 °C
Temperature: 29.55 °C
Temperature: 29.60 °C
Temperature: 29.42 °C
Temperature: 29.50 °C
```

Result

Sensor data collection and transfer using serial communication was successfully performed.

EX NO 3: Ultrasonic Sensor Interfacing with Arduino

Aim

To interface an ultrasonic sensor with Arduino UNO and measure the distance of an object using ultrasonic waves.

Apparatus Required

- Arduino UNO
- HC-SR04 Ultrasonic Sensor
- Breadboard
- Jumper Wires
- USB Cable
- Computer with Arduino IDE

Procedure

1. Connect the VCC pin of the HC-SR04 sensor to the 5V pin of Arduino.
2. Connect the GND pin of the sensor to the GND pin of Arduino.
3. Connect the Trigger pin to digital pin 11.
4. Connect the Echo pin to digital pin 8.
5. Open Arduino IDE and write the program.
6. Compile and upload the code to Arduino UNO.
7. Open the Serial Monitor.
8. Place an object in front of the sensor.
9. Observe the measured distance on the Serial Monitor.

Program

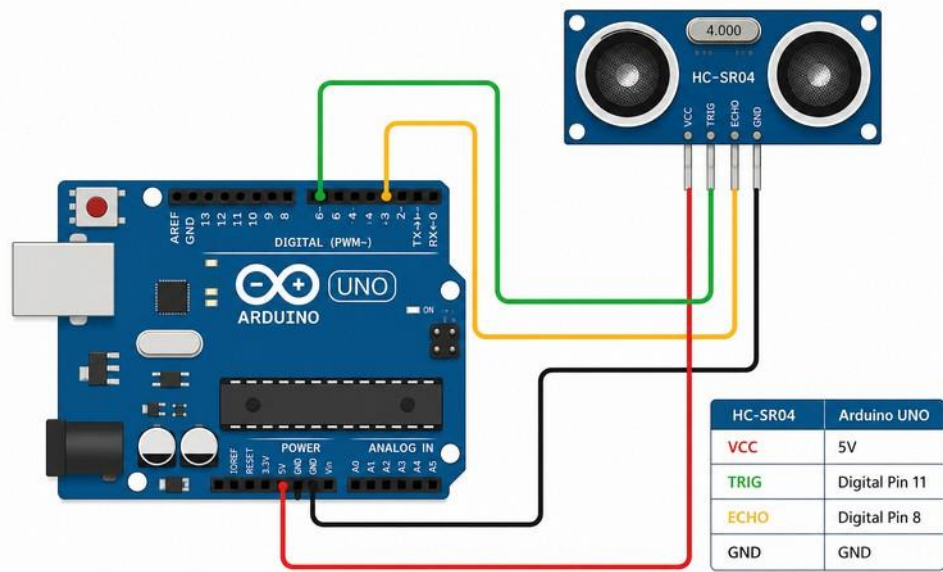
```
#define  
Trigger 11  
#define Echo  
8  
void setup  
{  
  Serial.begin(115200);  
  pinMode(Trigger, OUTPUT);  
  pinMode(Echo, INPUT);  
}  
void loop()  
{  
  digitalWrite(Trigger,
```

```

HIGH ); delayMicroseconds
(10); digitalWrite ( Trigger ,
LOW );
long duration = pulseIn ( Echo ,
HIGH ); int distance = duration /
58;
Serial . print (" Distance in
CM : "); Serial . println (
distance );
25 delay (1000) ;}

```

Circuit Diagram:



Output:

```

Distance in CM : 15
Distance in CM : 14
Distance in CM : 14
Distance in CM : 13
Distance in CM : 12
Distance in CM : 12
Distance in CM : 11
Distance in CM : 10

```

Result

The ultrasonic sensor was successfully interfaced with Arduino UNO, and the distance of the object was measured accurately.

EX NO 4: Traffic Light System using Arduino

Aim

To simulate a traffic light system using LEDs and Arduino UNO.

Apparatus Required

- Arduino UNO
- Red LED
- Yellow LED
- Green LED
- 220Ω Resistors
- Breadboard
- Jumper Wires
- USB Cable

Procedure

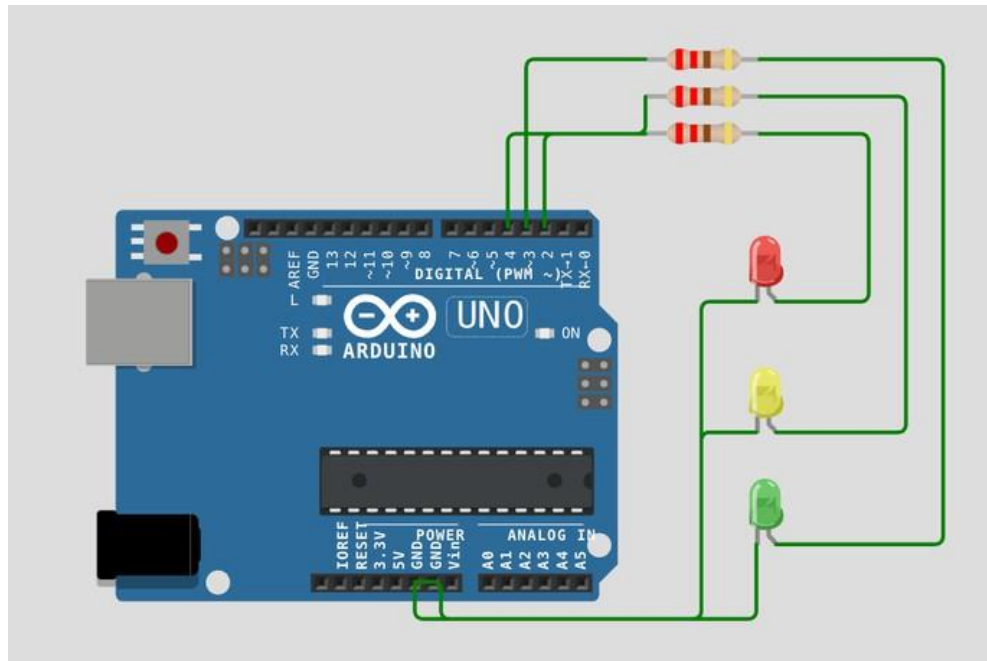
1. Connect the Red LED to pin 4 through a resistor.
2. Connect the Yellow LED to pin 2 through a resistor.
3. Connect the Green LED to pin 3 through a resistor.
4. Connect all LED cathodes to GND.
5. Upload the program to Arduino UNO.
6. Observe the sequential operation of LEDs.

Program

```
void setup () {  
  pinMode (2 , OUTPUT ) ; //  
  Yellow pinMode (3 ,  
  OUTPUT ) ; // Green  
  pinMode (4 , OUTPUT ) ; //  
  Red}  
void loop () {  
  digitalWrite (4 ,  
  HIGH ) ;  
  digitalWrite (3 ,  
  LOW ) ; delay  
  (2000) ;  
  digitalWrite (2 ,  
  HIGH ) ;  
  digitalWrite (4 ,  
  LOW ) ; delay
```

```
(2000) ;  
digitalWrite (3 ,  
HIGH ) ;  
digitalWrite (2 ,  
LOW ) ; delay  
(2000) ;}
```

Circuit Diagram:



Output:

Red LED : ON
Yellow LED : OFF
Green LED : OFF

Red LED : OFF
Yellow LED : ON
Green LED : OFF

Red LED : OFF
Yellow LED : OFF
Green LED : ON

Result

The traffic light system was successfully simulated using Arduino UNO and LEDs.

EX NO 5: Push Button with LED/Buzzer

Aim

To interface a push button with Arduino UNO and control an LED or buzzer.

Apparatus Required

- Arduino UNO
- Push Button
- LED or Buzzer
- 220Ω Resistor
- Breadboard
- Jumper Wires

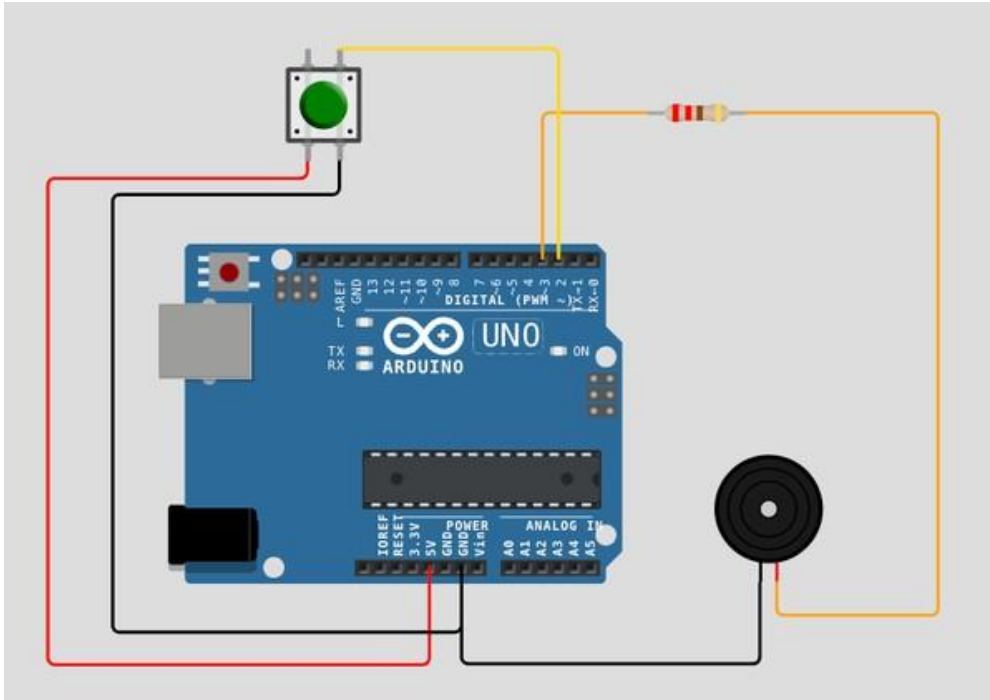
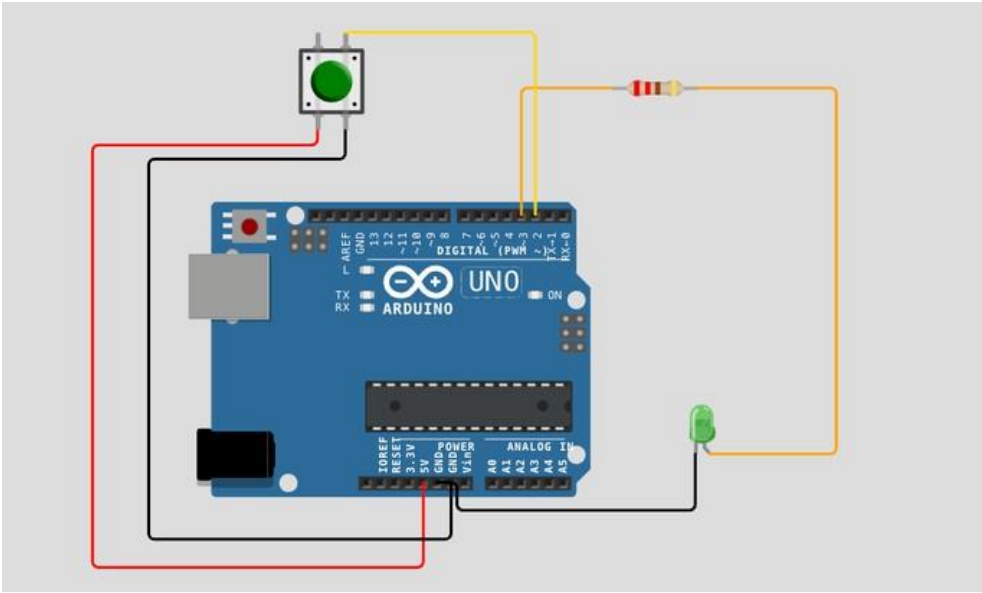
Procedure

1. Connect the push button to digital pin 2.
2. Connect the LED or buzzer to digital pin 3.
3. Upload the program to Arduino UNO.
4. Press the button and observe the output.

Program

```
const int buttonPin =  
2; const int ledPin =  
3; void setup () {  
  pinMode ( buttonPin , INPUT )  
  ; pinMode ( ledPin , OUTPUT )  
  ;} void loop () {  
  int buttonState = digitalRead ( buttonPin ) ;  
  if( buttonState == HIGH ) {  
    digitalWrite ( ledPin , HIGH ) ;  
    delay (300) ;  
    digitalWrite ( ledPin , LOW ) ;  
    delay (300) ;}  
  else {  
    digitalWrite ( ledPin , LOW ) ;}}
```

Circuit Diagram:



Output:

Push Button : NOT PRESSED
LED/Buzzer : OFF

Push Button : PRESSED
LED/Buzzer : ON

Push Button : PRESSED
LED/Buzzer : OFF

Push Button : NOT PRESSED
LED/Buzzer : OFF

Result

The push button was successfully interfaced with Arduino UNO, and the LED/Buzzer operated correctly.

EX NO 6: Potentiometer Controlled LED Brightness

Aim

To control LED brightness using a potentiometer and Arduino UNO.

Apparatus Required

- Arduino UNO
- LED
- 220Ω Resistor
- Potentiometer
- Breadboard and Jumper Wires

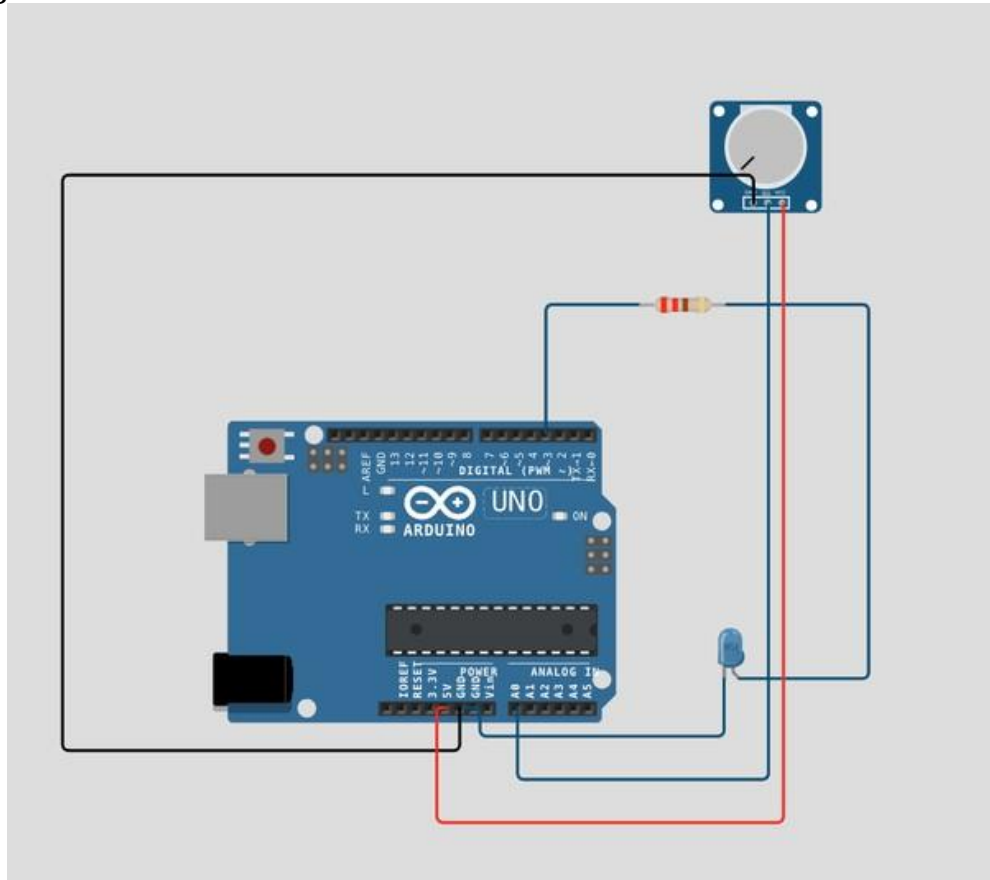
Procedure

1. Connect the potentiometer terminals to 5V and GND.
2. Connect the middle terminal to analog pin A0.
3. Connect the LED to pin 3 through a resistor.
4. Upload the program to Arduino UNO.
5. Rotate the potentiometer knob and observe LED brightness variation.

Program

```
const int analog_in  
= A0 ; void setup ()  
{  
  pinMode (3 ,  
  OUTPUT ) ; Serial  
  . begin (115200) ;}  
void loop () {  
  int val = analogRead ( analog_in ) ;  
  analogWrite (3 , val / 4) ;  
  Serial . print (" Potentiometer Value :  
  ") ; Serial . println ( val ) ;  
  delay (500) ;}
```

Circuit Diagram:



Output:

Potentiometer Value : 120

LED Brightness : LOW

Potentiometer Value : 450

LED Brightness : MEDIUM

Potentiometer Value : 780

LED Brightness : HIGH

Potentiometer Value : 1023

LED Brightness : MAXIMUM

Result

The LED brightness was successfully controlled using a potentiometer and Arduino UNO. The brightness varied according to the potentiometer position, and the corresponding analog values were displayed on the Serial Monitor.

EX NO 7: Handling and Setting up Raspberry Pi

Aim

To install and configure Raspberry Pi operating system.

Apparatus Required

- Raspberry Pi Foundation Raspberry Pi
- SD card
- Power adapter
- HDMI monitor
- Keyboard and mouse

Procedure

1. Download Raspberry Pi OS.
2. Flash the OS into SD card using Raspberry Pi Imager.
3. Insert SD card into Raspberry Pi.
4. Connect peripherals.
5. Power ON the board.
6. Complete initial setup.

Output

Raspberry Pi boots into Raspberry Pi OS desktop.

Result

Raspberry Pi setup and configuration was completed successfully.

EX NO 8: Working with Linux

Aim

To learn basic Linux commands in Raspberry Pi.

Apparatus Required

- Raspberry Pi
- Keyboard
- Monitor

Commands

- Pwd
- Ls
- mkdir test
- cd test
- touch file.txt
- cat file.txt
- nano file.txt
- python3

Procedure

1. Open terminal in Raspberry Pi.
2. Execute basic Linux commands.
3. Create and access directories/files.

Output

Linux terminal executes commands successfully.

Result

Basic Linux file system navigation was understood successfully.

EX.NO 9: Motor Control with Raspberry Pi

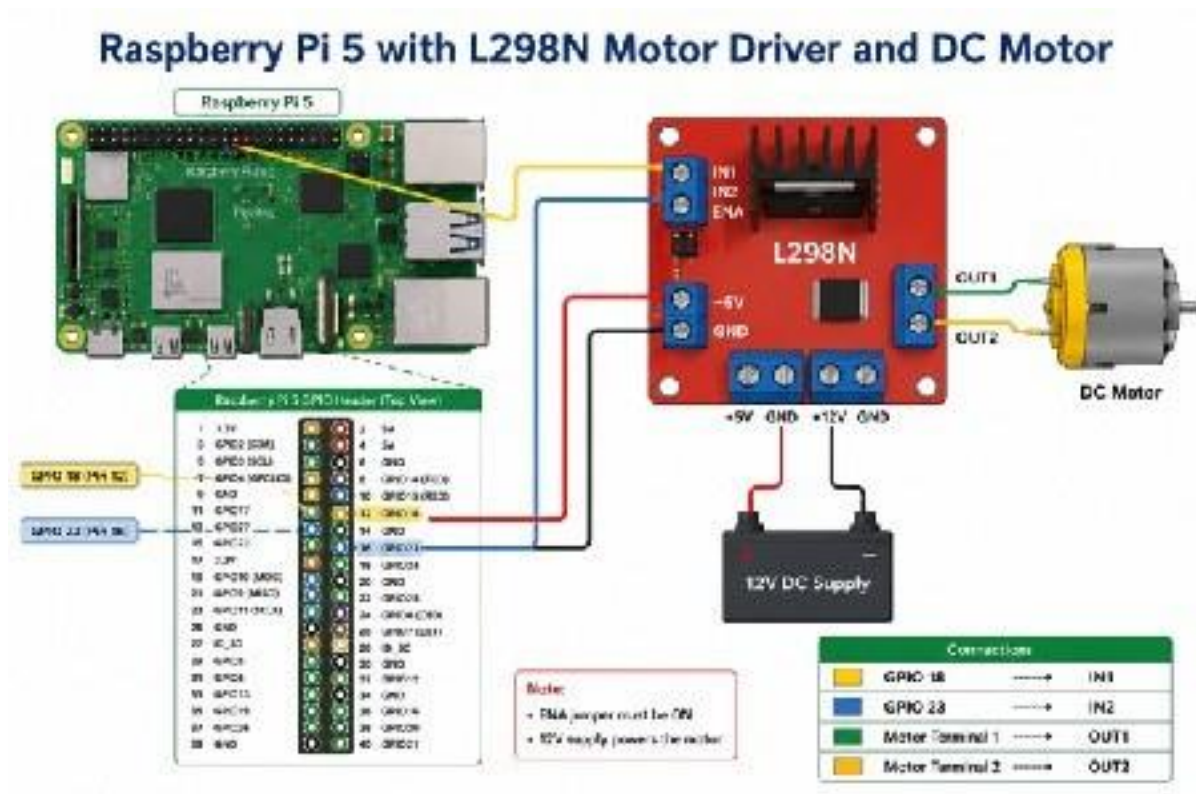
Aim

To control a DC motor using Raspberry Pi GPIO.

Apparatus Required

- Raspberry Pi
- L293D motor driver
- DC motor
- Jumper wires

Circuit Diagram



Program

```
sudo aptinstall python3-gpiozero
from gpiozero
import Motor from
7me import sleep

motor = Motor(forward=17,

backward=18) print("Motor Forward")
motor.forward()

sleep(5)

print("Mot
or Stop")
motor.stop
()
GPIO.clea
nup()
```

Output

Motor rotates in forward direction.

Result

DC motor control using Raspberry Pi GPIO was achieved successfully.

EX.NO 10: Controlling Ultrasonic Sensor with Raspberry Pi

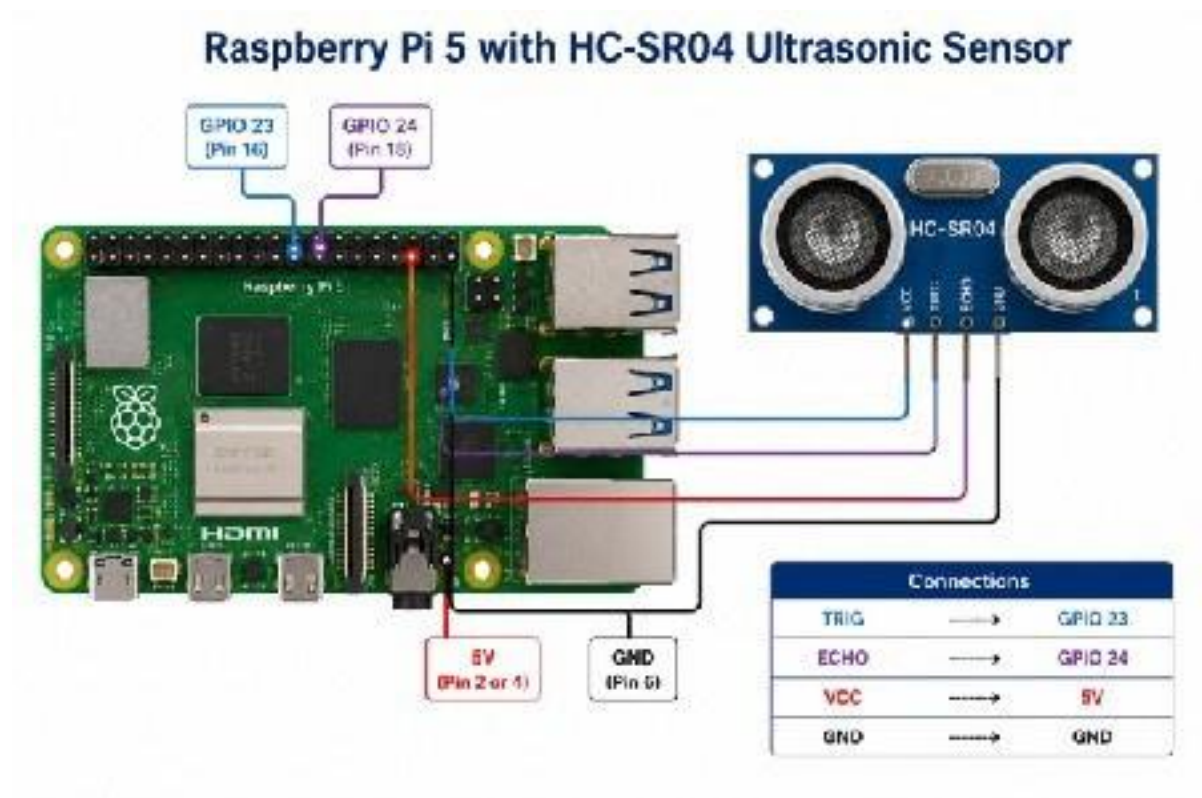
Aim

To measure distance using ultrasonic sensor and Raspberry Pi.

Apparatus Required

- Raspberry Pi
- HC-SR04 ultrasonic sensor
- Breadboard
- Jumper wires

Circuit Diagram



Procedure

1. Connect ultrasonic sensor to Raspberry Pi.
2. Execute the Python program.
3. Observe measured distance.

Program

```
pip install gpiozero
from gpiozero import
DistanceSensor from time
import sleep

sensor = DistanceSensor(echo=24,

trigger=23) while True:
distance = sensor.distance * 100

print("Distance :", round(distance, 2),
"cm") sleep(1)
```

Output

Distance value is displayed on terminal.

Result

Distance measurement using ultrasonic sensor was successfully implemented.

EX NO 11: Working with SQL

Aim

To perform basic SQL operations.

Apparatus Required

- MySQL Server
- Computer

Procedure

1. Open MySQL terminal.
2. Create database and table.
3. Insert values.
4. Retrieve records.

SQL Commands

1. Update the Raspberry Pi system using the terminal commands:
`sudo apt update`
`sudo apt upgrade -y`
2. Install MariaDB server on Raspberry Pi using:
`sudo apt install mariadb-server -y`
3. Start and enable the MariaDB service:
`sudo systemctl start mariadb`
`sudo systemctl enable mariadb`
4. Open the MariaDB terminal using:
`sudo mysql`
5. Create a new MariaDB user and provide privileges:
`CREATE USER 'piuser'@'localhost' IDENTIFIED BY '1234';`
`GRANT ALL PRIVILEGES ON *.* TO 'piuser'@'localhost';`
`FLUSH PRIVILEGES;`
6. Exit from MariaDB using:

EXIT;

7.Login again using the
created user: mysql -u
piuser -p

8. Create a new database:
CREATE DATABASE Raspberrypi;

9.Select the
database:
USE
Raspberrypi;

10. Create a table named
records: CREATE
TABLE records (
id INT AUTO_INCREMENT PRIMARY
KEY, name VARCHAR(100),
department
VARCHAR(100),
marks INT
);

11. Insert records into the table:
INSERT INTO records (name, department, marks)
VALUES
('Arun', 'ECE', 85),
('Kavin', 'CSE', 90),
('Priya', 'EEE', 88);

12. Display all
records using:
SELECT * FROM
records;

13. Update a
record using:
UPDATE records
SET marks =
95 WHERE
id = 1;

14. Delete a
record using:
DELETE FROM
records WHERE
id = 2;

15. Display the final
records table: SELECT *
FROM records;

16. Export the table into CSV
format: SELECT *
INTO OUTFILE
'/tmp/records.csv' FIELDS
TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
FROM records;

17. Exit from
MariaDB using:
EXIT;

18. View the exported CSV
file using: cat
/tmp/records.csv

Output:

```
+---+-----+-----+-----+
| id | name  | department | marks |
+---+-----+-----+-----+
| 1  | Arun  | ECE       | 85    |
| 2  | Kavın | CSE       | 90    |
| 3  | Priya | EEE       | 88    |
+---+-----+-----+-----+
```

Result

Basic SQL operations were performed successfully.

EX NO 12: Programming and Data Collection using Temperature Sensor

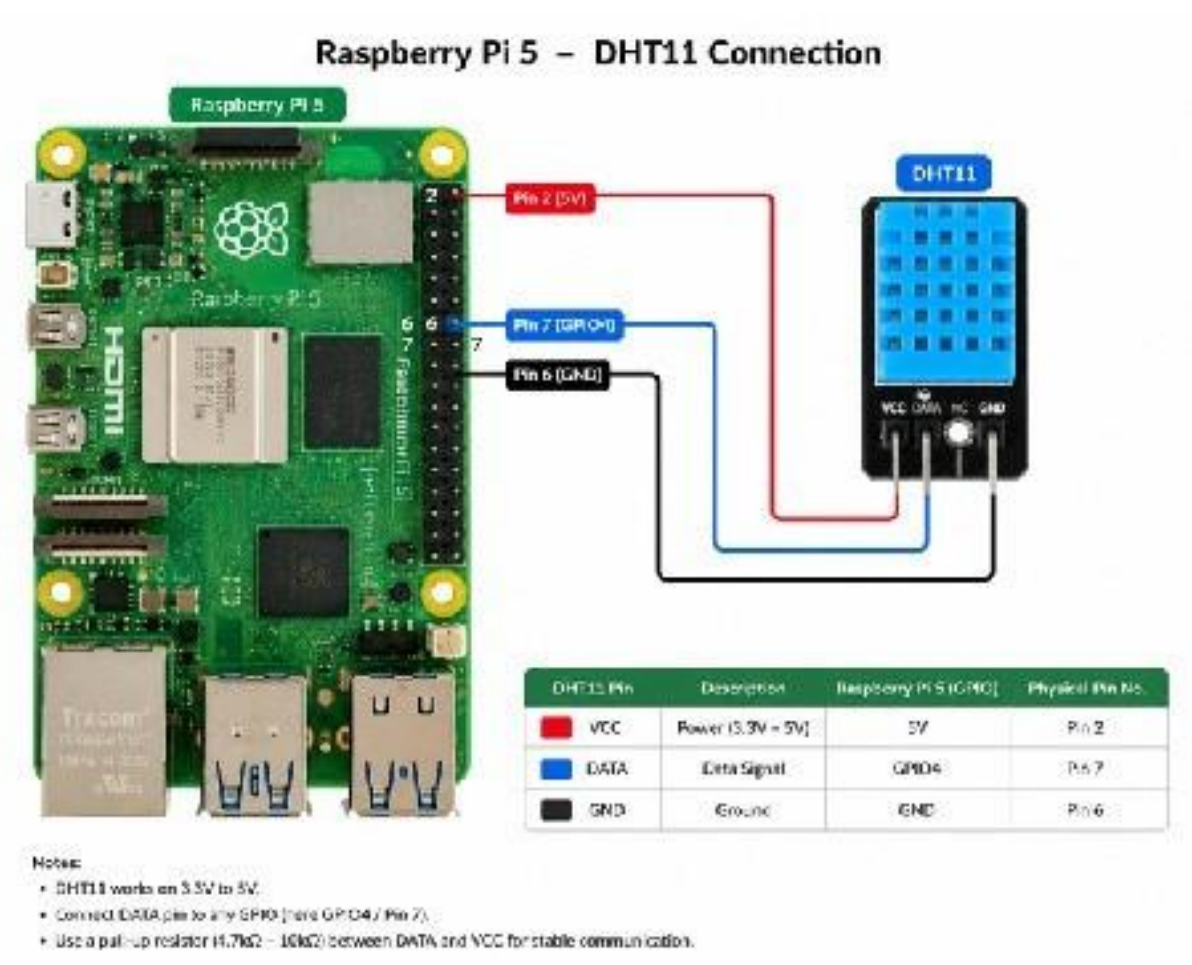
Aim

To collect temperature data using Raspberry Pi.

Apparatus Required

1. Raspberry Pi 5
2. DHT11 Sensor (Module preferred)
3. 3x Jumper Wires (Female-to-Female)

Circuit Diagram



Procedure

1. Connect temperature sensor.
2. Interface ADC with Raspberry Pi.
3. Execute Python program.
4. Observe readings.

Program

```
import
adafruit_dht
from time import
import board

import time
# Initialize DHT11 on GPIO 4 (Pin 7)
# For Pi 5, ensure the adafruit-circuitpython-dht library is
installed dht_device = adafruit_dht.DHT11(board.D4)

def get_data():
    try:
        # Read values from
        sensor temp =
        dht_device.temperature
        humidity =
        dht_device.humidity

        if temp is not None and humidity is not None:
            print(f"Temperature: {temp}°C | Humidity:
            {humidity}%")
        else:
            print("Failed to retrieve data")

    except RuntimeError as error:
        # Errors are common with DHT sensors, just keep trying
        print(f"Error: {error.args[0]}")
    except Exception
```

```
        as error:
        dht_device.exit(
        ) raiseerror

if __name__ == "__
main__":
    get_data()
```

Output:

Temperature: 29°C | Humidity: 68%
Temperature: 30°C | Humidity: 67%
Temperature: 29°C | Humidity: 69%
Temperature: 31°C | Humidity: 66%
Temperature: 30°C | Humidity: 68%

Result

Temperature data acquisition was successfully performed.

EX.NO 13: Fundamental Operations in Node-RED

Aim

To study basic Node-RED operations.

Apparatus Required

- Raspberry Pi
- Node-RED installed

Procedure

(Node-RED Flow Configuration)

Since Node-RED is a visual programming tool, the "program" consists of node properties:

- **Node 1 (Inject):**
 - *Payload:* Number: 1
 - *Topic:* LED ON
- **Node 2 (Inject):**
 - *Payload:* Number: 0
 - *Topic:* LED OFF
- **Node 3 (rpi-gpio out):**
 - *Pin:* 11 - GPIO17
 - *Type:* Digital Output
 - *Logic:* Direct (1 = High, 0 = Low)

Visual Logic:

[Inject: 1] --- >[rpi-gpio: Pin 11]

[Inject: 0] --- >[rpi-gpio: Pin 11]

Procedure

- **Hardware Connection:** Connect the LED's long leg (Anode) to **Pin 7** and the short leg (Cathode) through a resistor to **Pin 6 (GND)**.
- **Service Start:** Open the Linux terminal and run `node-red-start`.
- **Editor Access:** Open a web browser and navigate to `http://<your_pi_ip>:1880`.

- **Flow Assembly:**
 - Drag **two Inject nodes** and **one rpi-gpio out node** onto the Node-RED workspace.
 - Configure the **first Inject node** to send a **number 1**.
 - Configure the **second Inject node** to send a **number 0**.
 - Configure the **GPIO node** to **Physical Pin 7**.
 - **Wiring Connections:**
Connect the **output port** of both **Inject nodes** to the **input port** of the **rpi-gpio out node**.
 - **Deployment:**
Click the **Deploy** button located at the **top-right corner** of the Node-RED screen to apply the changes.

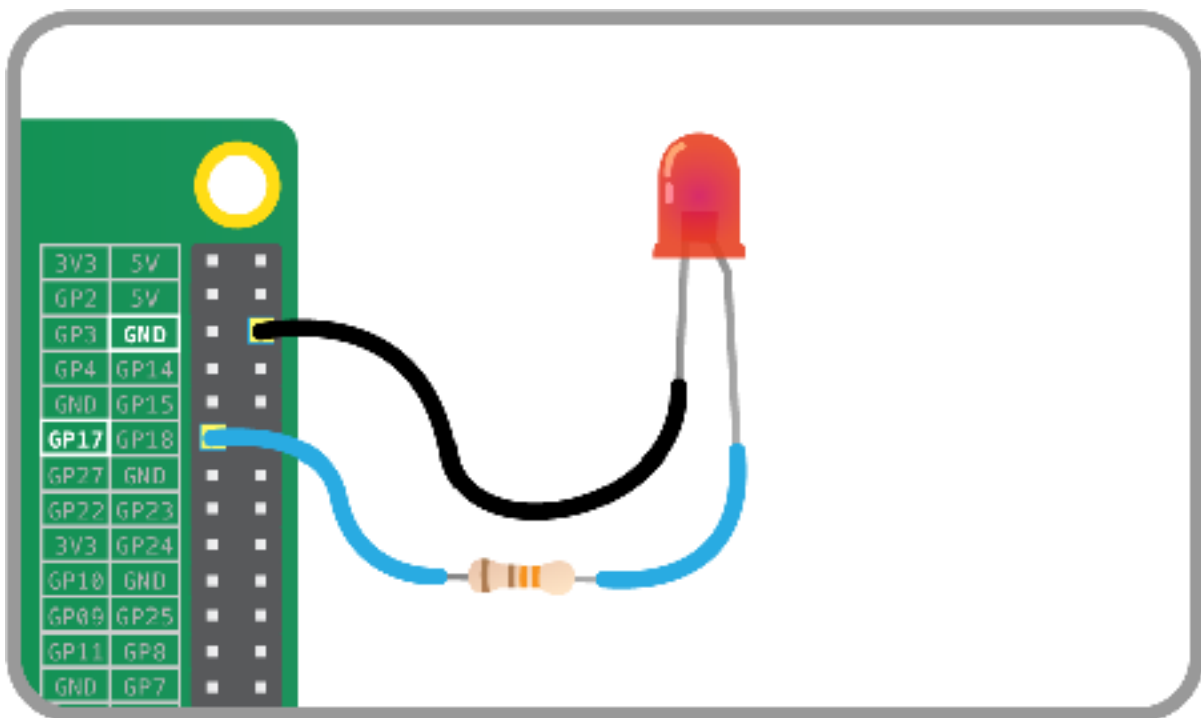
Working:

- Clicking the **Inject node (1)** turns the GPIO **ON**.
- Clicking the **Inject node (0)** turns the GPIO **OFF**.

Output

Upon clicking the button on the "LED ON" inject node, the LED successfully turns on.

Clicking the "LED OFF" inject node successfully turns the LED off. The system demonstrates real-time hardware control via a web-based flow interface.



Result

Basic Node-RED operations were successfully performed.

EX NO 14: Real Time Data Visualization in ThingsBoard

Aim

To visualize real-time sensor data in ThingsBoard.

Apparatus Required

- Raspberry Pi
- Sensors
- ThingsBoard platform

Procedure

1. Open ThingsBoard dashboard.
2. Add widgets and charts.
3. Connect telemetry data source.
4. Save dashboard.

Program

```
import adafruit_dht
import paho.mqtt.client as
mqtt import json

# ThingsBoard settings
BROKER =
"thingsboard.cloud"
PORT = 1883
ACCESS_TOKEN = "XskprV0aOvNc1xF21BJn"

# DHT11 sensor (GPIO4)
dht = adafruit_dht.DHT11(board.D4)

# MQTT client (modern style)
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.username_pw_set(ACCESS_TOKEN)
def on_connect(client, userdata, flags, reason_code, properties=None):
    print("Connected to ThingsBoard:", reason_code)
```

```

client.on_connect = on_connect
client.connect(BROKER, PORT, 60)
client.loop_start()
print("☐ Sending DHT11 data to ThingsBoard...")

while True: try:
    temperature =
    dht.temperature humidity =
    dht.humidity

    if temperature is not None and humidity is not
    None: payload = {
        "temperature":
        temperature,
        "humidity":
        humidity
    }

    client.publish(
        "v1/devices/me/telemet
        ry",
        json.dumps(payload)
    )

    print("Sent:", payload)

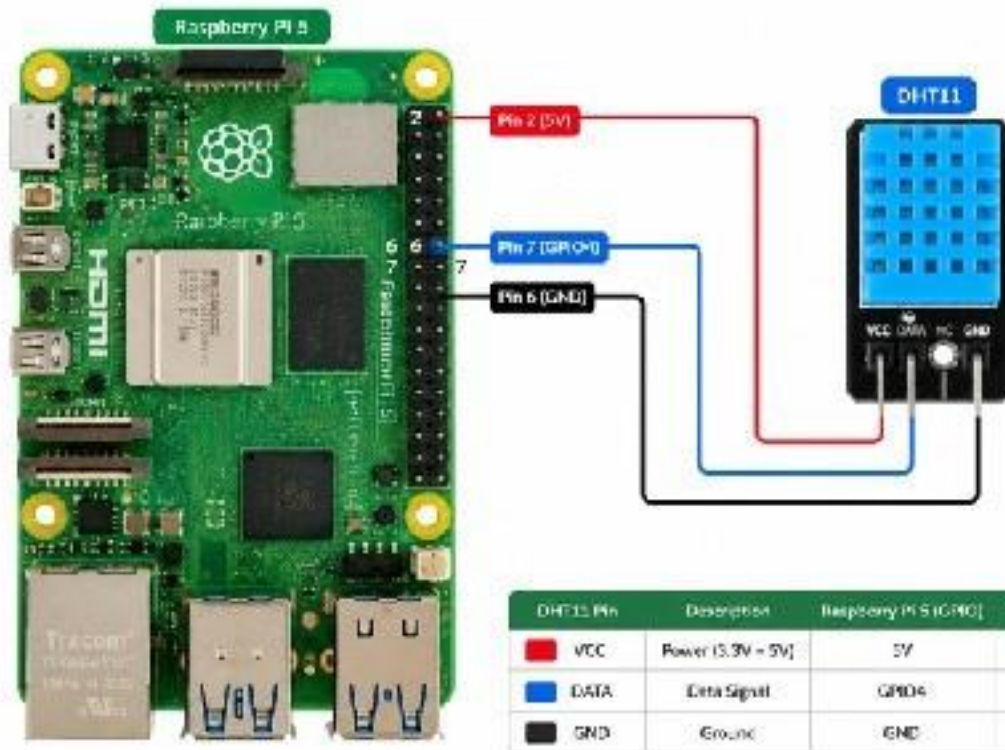
else:
    print("Sensor read failed")

except RuntimeError as e:
    print("Retrying sensor...",
        e.args[0])

time.sleep(5)

```

Raspberry Pi 5 – DHT11 Connection



Notes:

- DHT11 works on 3.3V to 5V.
- Connect DATA pin to any GPIO (here GPIO4 / Pin 7).
- Use a pull-up resistor (4.7kΩ ~ 10kΩ) between DATA and VCC for stable communication.

Output:

Connected to ThingsBoard: Success

🔗 Sending DHT11 data to ThingsBoard...

```
Sent: {'temperature': 29, 'humidity': 68}
Sent: {'temperature': 30, 'humidity': 67}
Sent: {'temperature': 29, 'humidity': 69}
Sent: {'temperature': 31, 'humidity': 66}
Sent: {'temperature': 30, 'humidity': 68}
```

Result

Real-time sensor data visualization was successfully implemented using ThingsBoard.